

Activities

1) Find 10 different model providers and their models released in last 12 months

Model Provider	Model	Release Date	Cutoff Date	Cost of Tokens	Context Window	Type	When to Use
Anthropic	Claude Fable 5	Jun 2026	Apr 2026	Mid-tier (≈\$3 per 1M tokens)	200K	Closed	Best for reasoning, coding, agentic workflows
Anthropic	Claude Opus 4.8	Jul 2026	Apr 2026	Premium (≈\$15 per 1M tokens)	200K	Closed	Deep analysis, enterprise QA, automation
OpenAI	GPT-5.5 Pro	Apr 2026	Mar 2026	High (≈\$20 per 1M tokens)	128K	Closed	Complex reasoning, research-heavy tasks
OpenAI	GPT-5.6	Jul 2026	May 2026	Premium (≈\$18 per 1M tokens)	256K	Closed	Long-context workflows, enterprise integration
Google DeepMind	Gemini 3.2 Pro	Jul 2026	May 2026	Competitive (≈\$10 per 1M tokens)	2M	Closed	Multimodal (text, vision, audio), long docs
DeepSeek	DeepSeek V4 Flash	Early 2026	Jan 2026	Low-cost (≈\$2 per 1M tokens)	128K	Closed	Browsing, agentic coding, efficiency
Zhipu AI	GLM-5.2	Apr 2026	Feb 2026	Open-weight (free/commercial license)	128K	Open	Custom deployments, transparency,

							enterprise QA
xAI	Grok 4	2026	Apr 2026	Mid-tier (≈\$8 per 1M tokens)	2M	Closed	Very large context tasks, reasoning chains
Meta	LLaMA 3.1	2025–2026	Ongoing	Free (open-source)	128K	Open	Research, fine-tuning, academic use
Mistral AI	Mixtral 8x22B	2025–2026	Ongoing	Free/commercial license	64K	Open	Efficient coding, open-source enterprise

Notes

- **Closed models (Anthropic, OpenAI, Google, DeepSeek, xAI)** → Best raw performance, but higher costs and limited customization.
- **Open models (GLM, LLaMA, Mixtral)** → Free or low-cost, customizable, but slightly weaker in frontier reasoning.
- **Context windows** → Grok 4 and Gemini 3.2 Pro lead with **2M tokens**, ideal for analysing huge datasets or documents.
- **Enterprise QA relevance** → Claude Opus 4.8, GPT-5.5 Pro, and Gemini 3.2 Pro are most widely adopted in testing/project management workflows.

the most practical models for enterprise QA and project workflows are **Claude Opus 4.8** (reasoning + automation), **GPT-5.5 Pro** (deep analysis), and **Gemini 3.2 Pro** (multimodal + long context).

2) Read 2–3 articles about how transformer works?

Goal to understand:

- (a) Self-attention mechanism
- (b) Vector embedding
- (c) Feed forward Neural Network

Here are three articles that explain how transformers work, focusing on self-attention, vector embeddings, and feed-forward neural networks.

1. Attention Mechanisms: How Transformers Actually Work (Hakia, July 2026)

- **Focus:** Deep dive into **self-attention** and multi-head attention.
- **Key Points:**
 - Self-attention computes relationships between all tokens in parallel.
 - Uses **Query (Q), Key (K), Value (V)** transformations to decide relevance.
 - Multi-head attention allows the model to capture different relationships simultaneously.
 - Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Takeaway:** Self-attention lets each word “look at” every other word, enabling rich contextual understanding.

References:

- **Article:** *Attention Is All You Need* (Vaswani et al., 2017)
- **Source:** arXiv (Google Brain & Google Research)
- **Link:** [Read on arXiv \(PDF\)](#)

2. Transformers from First Principles: Part 2 – Embeddings (Matías Battaglia, March 2026)

- **Focus:** Explains **vector embeddings** in detail.
- **Key Points:**
 - Token IDs are arbitrary; embeddings map them into meaningful **dense vectors**.

- Embeddings are learned during training so that semantically similar words (e.g., “cat” and “dog”) are close in vector space.
- Combines **token embeddings** (what the word is) with **positional embeddings** (where it appears).
- Embedding dimension (e.g., 768, 1024) determines how much information each token carries.
- **Takeaway:** Embeddings transform discrete tokens into continuous vectors that encode semantic meaning and position.

References

- **Article:** *Transformers from First Principles: Part 2 – Embeddings* (Matías Battaglia, 2026)²
- **Source:** Personal blog / technical deep dive
- **Link:** Read here (matiasbattaglia.com in Bing)

3. Feed-Forward Networks in Transformers: Architecture, Parameters & Efficiency (Michael Brenndoerfer, June 2025)

- **Focus:** Explains the **feed-forward neural network (FFN)** inside each transformer block.
- **Key Points:**
 - Attention is linear; FFN adds **nonlinearity** and per-token computation.
 - Standard design: **two-layer MLP** with expansion (4× hidden dimension) and compression back to original size.
 - Formula:

$$\text{FFN}(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2$$

- Accounts for most parameters in a transformer (often >60%).
- Provides the “thinking” step after attention routes information.
- **Takeaway:** FFNs transform contextual representations into richer abstractions, enabling complex language understanding.

References

- **Article:** *Attention Is Not All You Need: The Importance of Feedforward Networks in Transformer Models* (Isaac Gerber, 2025)

- Source: arXiv
- Link: [Read on arXiv](#)

How These Fit Together

- **Self-Attention:** Finds relationships between tokens (context integration).
- **Vector Embeddings:** Provide the numerical representation of tokens (semantic meaning + position).
- **Feed-Forward Networks:** Apply nonlinear transformations to each token's enriched representation (feature extraction).

Together, these three components form the backbone of the transformer architecture that powers modern LLMs like GPT-5, Claude, and Gemini.